Project Report: Anime Recommendation System

Hediyeh Nayebi

July 6, 2026

1 Introduction

In the contemporary digital era, the proliferation of online content has necessitated the development of sophisticated recommendation systems to assist users in navigating vast information spaces. Among these, the anime domain presents a unique challenge due to the diverse preferences of the user base and the hierarchical nature of anime metadata.

This project aims to design and implement a robust recommendation engine tailored for the anime dataset. The core objective is to move beyond simple popularity-based suggestions by leveraging Collaborative Filtering (CF) techniques. By analyzing user-item interaction patterns—specifically rating behaviors—we aim to uncover latent features that drive user preferences.

The primary contributions of this report are threefold:

1. **Data Exploration:** A comprehensive analysis of user-item interaction matrices to understand sparsity and interaction distributions.
2. **Noise Reduction:** Implementation of data preprocessing techniques to mitigate the "cold-start" problem and improve dataset quality.
3. **Model Evaluation:** Development and assessment of recommendation algorithms, evaluated against standard metrics such as Root Mean Square Error (RMSE) to ensure prediction accuracy.

Through this systematic approach, we intend to provide a scalable and effective solution capable of delivering personalized recommendations within the high-dimensional anime ecosystem.

2 Data Exploration and Matrix Analysis

To understand the dataset structure, we performed an initial analysis of the user-item interaction matrix. The following Python code was used to calculate the matrix density and sparsity:

Listing 1: Calculating Matrix Sparsity

```
# Calculate dimensions
num_users = rating_df['user_id'].nunique()
num_animes = rating_df['anime_id'].nunique()
total_ratings = len(rating_df)

# Sparsity calculation
density = total_ratings / (num_users * num_animes)
sparsity = 1 - density

print(f"Matrix Sparsity: {sparsity * 100:.2f}%")
```

2.1 Visualizing the Long Tail

We visualized the interaction frequency to identify the "Long Tail" phenomenon, where a small number of anime titles receive a disproportionate number of ratings.

Listing 2: Plotting Interaction Frequency

```
# Plotting the Long Tail distribution
anime_popularity = rating_df.groupby('anime_id')['
    ↪ rating'].count()
plt.plot(np.sort(anime_popularity.values))
plt.yscale('log') # Log scale to handle the skewness
plt.title('Anime Interaction Frequency (Long Tail)')
plt.show()
```

2.2 Mathematical Context

The interaction dataset is modeled as a matrix $R \in \mathbb{R}^{U \times I}$, where U denotes the set of users, I denotes the set of anime titles, and r_{ui} represents the rating given by user u to anime i.

Given that most users interact with only a tiny fraction of the available anime, R is extremely sparse. The sparsity is mathematically defined as:

$$\text{Sparsity} = 1 - \frac{|\{r_{ui} \mid r_{ui} \neq 0\}|}{|U| \times |I|} \tag{1}$$

Furthermore, the interaction frequency visualized in the long-tail plot follows a power-law distribution. This is expressed as $P(x) \propto x^{-\alpha}$, indicating that a highly skewed minority of items receives the vast majority of interactions.

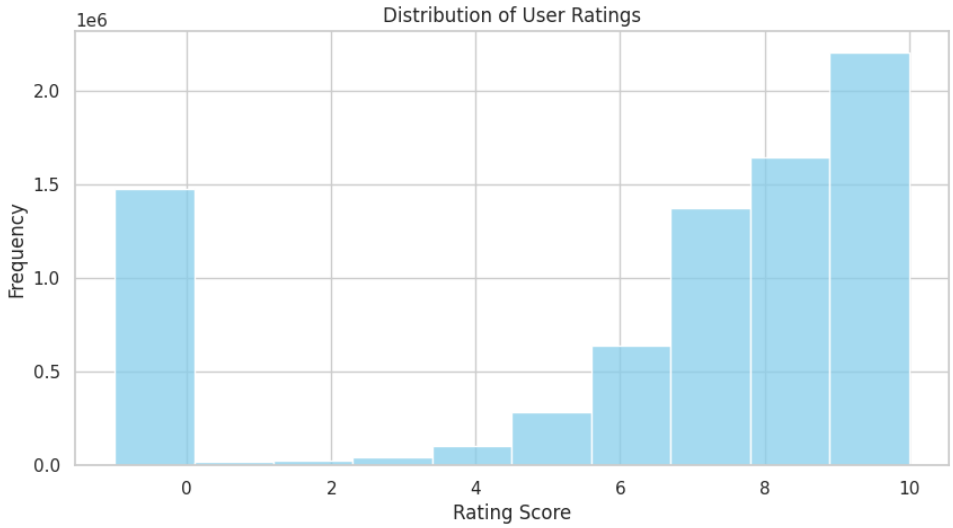

Figure 1: Distribution of User Ratings

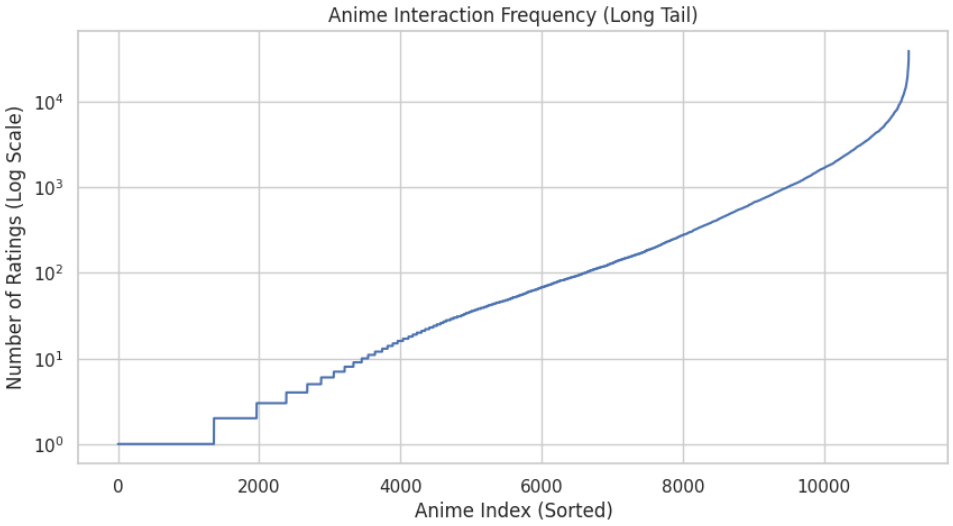

Figure 2: Long Tail Frequency (Logarithmic Scale)

3 Data Preprocessing & Noise Reduction

To address the cold-start problem and mitigate the computational challenges imposed by the high sparsity of the user-item interaction matrix, a robust data filtering pipeline was implemented. In collaborative filtering systems, users with very few interactions often introduce high noise and minimal signal, which severely limits the model's ability to learn stable and reliable preference profiles.

Therefore, an activity threshold τ is defined such that any user u with a total number of interactions less than τ is excluded from the active dataset. Formally, let $R_u = \{i \in I \mid r_{ui} \neq 0\}$ represent the set of items rated by user u. The filtering criterion for the active user base is expressed as:

$$U_{\text{filtered}} = \{u \in U \mid |R_u| \geq \tau\}$$

For this implementation, the threshold was set to $\tau = 50$. This preprocessing step ensures that the recommendation engine trains exclusively on high-quality user profiles with well-established historical preferences, thereby significantly reducing matrix sparsity and optimizing computational efficiency.

The quantitative impact of this filtering process on the dataset dimensions is summarized in Table 1.

Table 1: Dataset Statistics Before and After Noise Reduction ($\tau = 50$)

Metric	**Original Dataset**	**Filtered Dataset**	**Impact (% Change)**
Total Ratings	7,813,737	7,166,253	-8.29%
Unique Users	73,515	39,466	-46.32%

4 User-Item Matrix Construction

Following the noise reduction phase, the longitudinal dataset was transformed into a wide-format user-item interaction matrix $R \in \mathbb{R}^{U \times I}$, where rows correspond to unique users and columns to anime titles. Each element r_{ui} represents the rating given by user u to anime i.

In cases of duplicated interactions (e.g., a user rating the same anime multiple times due to data logging anomalies), the arithmetic mean of the duplicate ratings was computed to aggregate the preferences into a single representative value. Finally, all unobserved interactions were explicitly imputed with a value of 0. This structural transformation is a fundamental prerequisite for computing vector-based similarities in collaborative filtering architectures.